

Università degli Studi di Salerno
Corso di Laurea in Informatica

Machine Learning

Anno Accademico 2025/2026

Report di progetto

CARVAL

L'intelligenza artificiale al servizio della valutazione delle auto usate

Autore

Ferdinando Ranieri

Docenti

Prof. Giuseppe Polese
Prof.ssa Loredana Caruccio

Indice

Indice.....	2
1. Scenario, problema e task analizzato	3
1.1 Da un'auto in panne a un problema di Machine Learning.....	3
1.2 Il problema affrontato	3
1.3 Perché Carval?	3
2. Il dataset utilizzato e le sue caratteristiche	4
2.1 Da dove arrivano i dati.....	4
2.2 Cosa contiene il dataset.....	4
2.3 Una correlazione (troppo) perfetta	4
2.4 Cosa raccontano i dati.....	5
3. Issue analizzate, soluzioni proposte e scelte di progettazione	8
3.1 Il rumore nei dati: quando la stessa auto ha mille nomi	8
3.2 Feature ridondanti o pericolose	8
3.3 I valori mancanti: colmare senza inventare.....	8
3.4 Numeri che il modello possa capire: encoding e scaling	9
3.5 Outlier: quando un dato è troppo per essere vero.....	9
3.6 Feature engineering e selezione delle feature	9
3.7 Il problema dello squilibrio tra classi	10
3.8 Perché il Random Forest?	11
4. Analisi delle prestazioni	12
4.1 Come si valuta un modello, davvero.....	12
4.2 Risultati – task di classificazione	12
4.3 Risultati – task di regressione	14
4.4 Dove (e perché) il modello sbaglia.....	15
Classificazione	15
Regressione	15
5. Considerazioni finali e sviluppi futuri	16

1. Scenario, problema e task analizzato

1.1 Da un'auto in panne a un problema di Machine Learning

Carval nasce da una situazione fin troppo comune: un'auto che si guasta irreparabilmente e la necessità di comprarne una usata senza avere alcun criterio oggettivo per giudicare se il prezzo richiesto sia corretto. La ricerca sul mercato dell'usato ha messo in luce un problema noto a chiunque ci sia passato: valutazioni molto diverse tra concessionari e privati, annunci quasi identici con prezzi che possono variare anche di migliaia di euro, e nessun modo semplice per l'acquirente (o il venditore) di orientarsi.

Da qui l'idea: costruire un sistema di Machine Learning capace di imparare da un ampio storico di transazioni reali e restituire una stima oggettiva del valore di un'auto usata, riducendo l'asimmetria informativa tra chi vende e chi compra.

Come per qualunque applicazione di AI su dati reali, il problema porta con sé due sfide di fondo che hanno guidato molte delle scelte descritte in questo documento: la qualità e la disponibilità dei dati, che condiziona direttamente le prestazioni ottenibili, e l'explainability, cioè la necessità di capire (e poter spiegare) perché il modello arriva a una certa stima.

1.2 Il problema affrontato

L'obiettivo è sviluppare un modello di apprendimento supervisionato che predica il prezzo di un'auto usata sulla base delle sue caratteristiche, sfruttando l'esperienza maturata da un ampio dataset storico. Per sperimentare più a fondo le tecniche viste e non viste a corso, si è scelto di affrontare il problema da due prospettive complementari:

- regressione: il modello predice il prezzo esatto del veicolo;
- classificazione: il prezzo viene discretizzato in fasce di valore (es. 0–5.000€, 5.000–10.000€, ecc.) e il modello impara ad assegnare l'auto alla fascia corretta.

Affrontare lo stesso problema con due formulazioni diverse permette di confrontare tecniche e metriche di valutazione differenti, dimostrando padronanza di entrambe le famiglie di approcci.

1.3 Perché Carval?

L'idea alla base del progetto è semplice: rendere più democratico l'accesso a valutazioni affidabili. Carval è pensato per rispondere alle esigenze di più tipologie di utenti: acquirenti di auto usate, che possono verificare se il prezzo richiesto è in linea con il mercato; venditori privati e concessionari, che possono ottenere una stima realistica per fissare un prezzo competitivo; compagnie assicurative, che possono impiegare il sistema per stimare il valore residuo dei veicoli.

A supporto pratico del modello è stata inoltre sviluppata una semplice interfaccia web (Streamlit) che permette di inserire le caratteristiche di un veicolo e ottenere una stima immediata del suo valore. Va detto chiaramente: non si tratta di una quotazione ufficiale, ma di un supporto alla decisione, costruito su pattern appresi da dati storici.

2. Il dataset utilizzato e le sue caratteristiche

2.1 Da dove arrivano i dati

Il dataset è stato reperito dalla piattaforma Kaggle (“Used Car Auction Prices”, <https://www.kaggle.com/datasets/tunguz/used-car-auction-prices>). La scelta di partire da un dataset pubblico già strutturato, anziché raccogliere dati autonomamente tramite web scraping, è motivata da ragioni molto pratiche: garantisce la riproducibilità degli esperimenti ed evita le problematiche legali e di manutenzione tipiche di una raccolta dati autonoma, che avrebbe richiesto tempo e risorse difficilmente giustificabili per un progetto di questa portata.

Questo non significa che il dataset sia perfetto: copre esclusivamente il mercato nord-americano fino al 2015, quindi il modello è addestrato su informazioni storiche e non cattura necessariamente le dinamiche di mercato più recenti o quelle di altri paesi. È un limite di cui tenere conto nell'interpretare i risultati, non un difetto nascosto.

2.2 Cosa contiene il dataset

Il dataset di partenza è costituito da 558.811 osservazioni descritte da 16 caratteristiche, tra cui: anno di produzione, marca, modello, allestimento, tipo di carrozzeria, trasmissione, VIN, stato di registrazione, venditore, condizione del veicolo, chilometraggio, colore esterno e interno, MMR (valore di mercato storico), prezzo di vendita (target) e data di vendita.

Le principali statistiche descrittive delle variabili numeriche sono riassunte in Tabella 1.

Caratteristica	Count	Media	Std	Min	Max
year	558.811	2010,04	3,97	1982	2015
condition	547.017	3,42	0,95	1,0	5,0
odometer	558.717	68.323,20	53.397,75	1,0	999.999
mmr	558.811	13.769,32	9.679,87	25,0	182.000
sellingprice	558.811	13.611,26	9.749,66	1,0	230.000

Tabella 1: statistiche descrittive delle caratteristiche numeriche del dataset originale.

2.3 Una correlazione (troppo) perfetta

La matrice di correlazione (Figura 1) è uno dei primi strumenti utilizzati per capire come le variabili si relazionano tra loro, ed è anche il punto in cui è emerso il problema più insidioso di questo dataset.

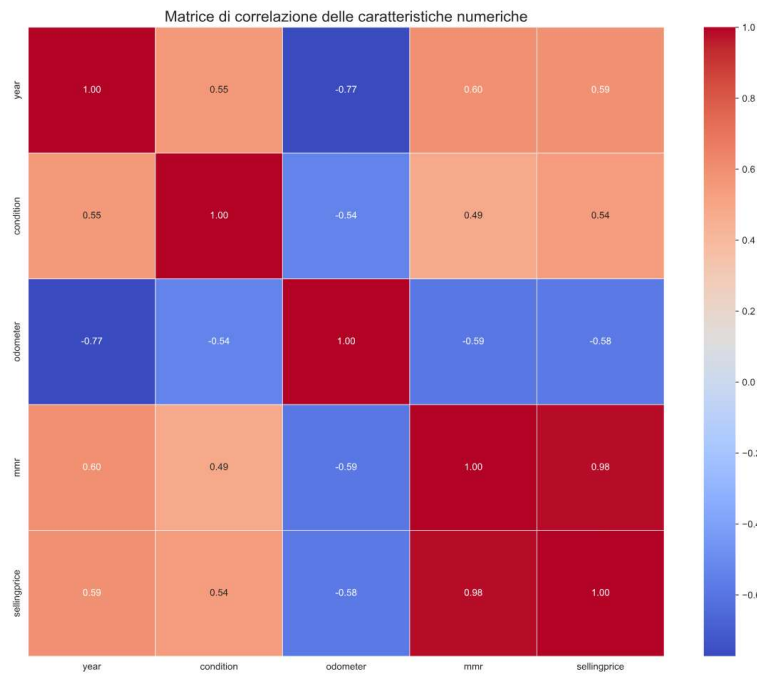


Figura 1: matrice di correlazione delle caratteristiche numeriche.

La variabile mmr correla al 98% con il target sellingprice: un valore così alto non è una buona notizia come sembrerebbe, ma il sintomo di un rischio di data leakage. mmr è infatti un indicatore di mercato calcolato con informazioni sostanzialmente coincidenti con il prezzo di vendita, quindi non sarebbe disponibile in una situazione reale di stima a priori: usarlo avrebbe significato costruire un modello che sembra ottimo in fase di addestramento ma inutile nella pratica. Per questo motivo la caratteristica è stata esclusa fin da subito dal set di feature utilizzate per l'addestramento.

2.4 Cosa raccontano i dati

Di seguito sono riportate le distribuzioni delle principali variabili numeriche del dataset.

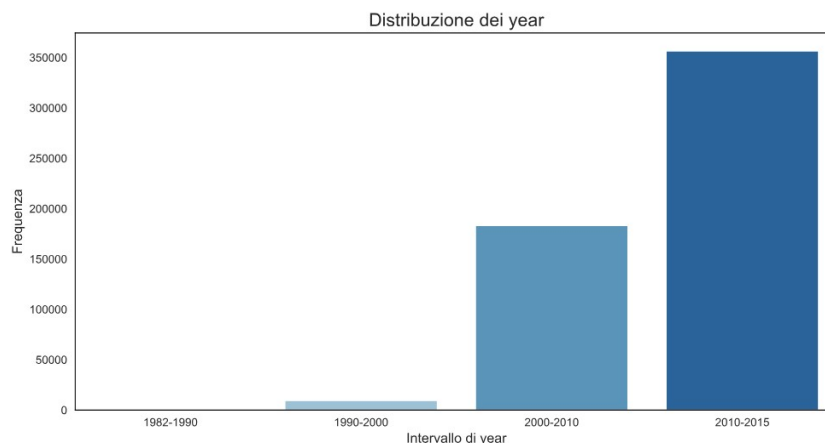


Figura 2a: distribuzione dell'anno di produzione.

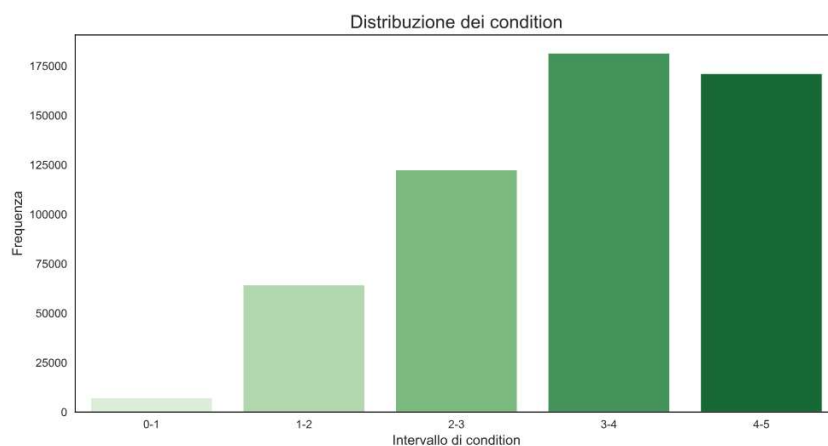


Figura 2b: distribuzione della condizione del veicolo.

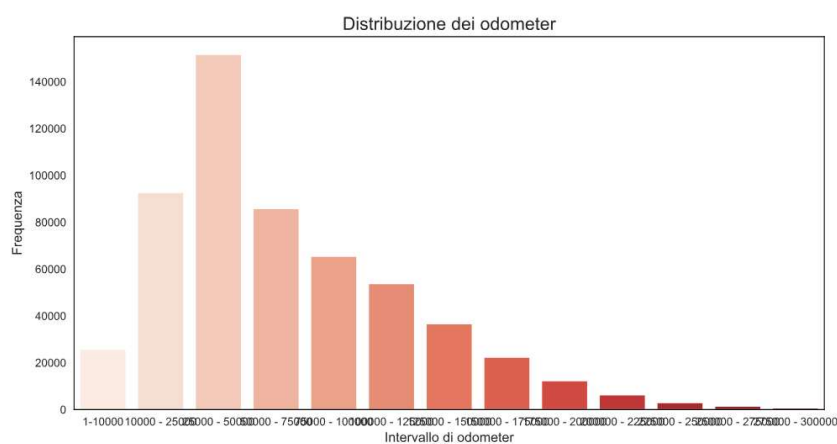


Figura 2c: distribuzione del chilometraggio.

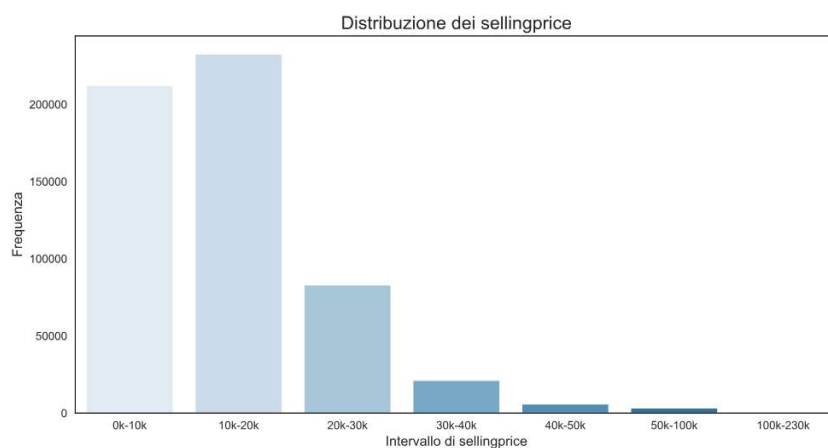


Figura 2d: distribuzione del prezzo di vendita (target).

Guardando invece come il prezzo varia rispetto ad alcune variabili chiave emergono pattern di mercato piuttosto intuitivi, ma utili da confermare numericamente prima di passare alla modellazione.

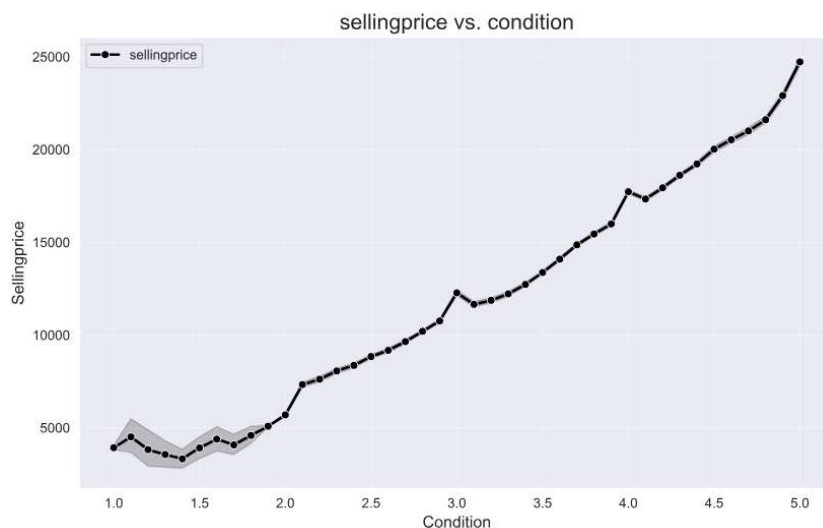


Figura 3a: condizione del veicolo vs prezzo di vendita.

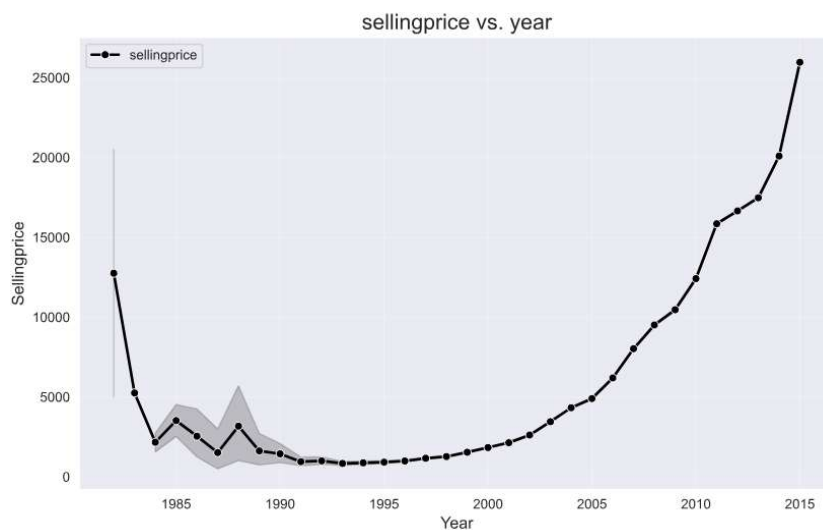


Figura 3b: anno di produzione vs prezzo di vendita.

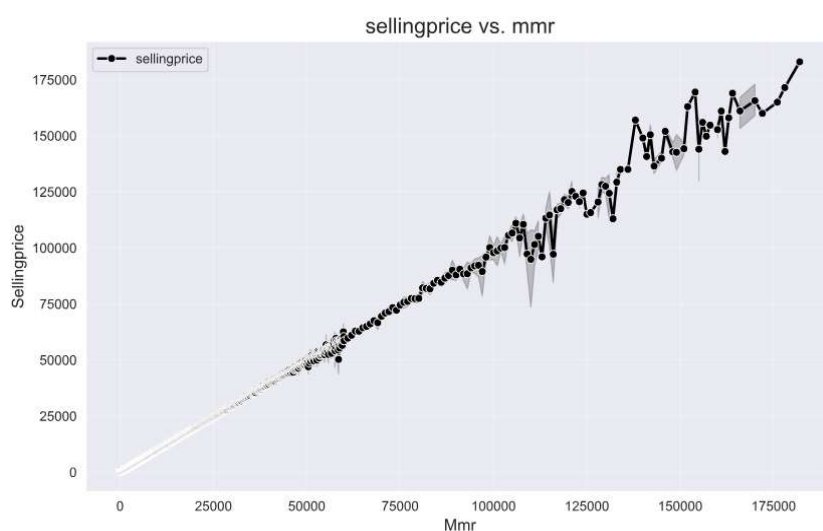


Figura 3c: mmr vs prezzo di vendita.

Dalla Figura 3a emerge che il prezzo cresce al crescere del punteggio di condizione, come ci si aspetterebbe. Dalla Figura 3b si osserva un andamento crescente a partire dal 1995. Dalla Figura 3c la relazione tra mmr e prezzo risulta pressoché coincidente con la bisettrice del piano: la conferma visiva, oltre che numerica, del rischio di data leakage discusso poco sopra.

3. Issue analizzate, soluzioni proposte e scelte di progettazione

3.1 Il rumore nei dati: quando la stessa auto ha mille nomi

Un'analisi preliminare ha evidenziato diverse fonti di rumore nei dati grezzi, del tipo che si incontra praticamente sempre lavorando con dati raccolti “dal mondo reale”. Uno stesso marchio automobilistico compariva sotto denominazioni diverse (es. ‘mercedes’, ‘mercedes-b’, ‘mercedes-benz’): tali varianti sono state consolidate in un'unica forma standard tramite un mapping esplicito. In alcuni campioni il valore di allestimento (trim) coincideva col valore di modello: in questi casi il valore è stato sostituito con la stringa ‘base’. Infine, i valori poco informativi nelle variabili di colore (es. il simbolo “-”) sono stati trattati come dati mancanti.

La variabile trim presentava inoltre un'elevata cardinalità e variabilità lessicale: troppi valori diversi che, di fatto, indicavano varianti molto simili tra loro. I valori sono stati quindi raggruppati in categorie semantiche più ampie (base, special edition, sport, touring, luxury, other). Un raggruppamento analogo è stato applicato alla variabile body (sedan, suv, hatchback, van, coupé, cabriolet, station wagon, pickup, other), riducendo la dimensionalità senza perdere informazione rilevante.

3.2 Feature ridondanti o pericolose

Non tutte le informazioni disponibili aiutano il modello: alcune sono semplicemente inutili, altre possono addirittura peggiorare le prestazioni introducendo rumore o falsi pattern. Sono state quindi rimosse le seguenti caratteristiche:

- mmr, per il rischio di data leakage discusso in precedenza;
- vin, codice univoco che non consente al modello di apprendere pattern generalizzabili;
- seller, ritenuto non informativo ai fini della predizione del prezzo;
- saledate, poiché non si è scelto di trattare esplicitamente l'informazione temporale;
- state, per garantire che il modello non sia vincolato ai confini geografici di registrazione.

A questo si aggiunge un filtro di coerenza: sono stati esclusi i campioni con prezzo inferiore a 500€ (valori non realistici, probabilmente errori di inserimento) e le marche con meno di 500 osservazioni complessive, troppo poco rappresentate per permettere al modello di imparare pattern affidabili.

3.3 I valori mancanti: colmare senza inventare

La Figura 4 mostra la quantità di valori nulli per ciascuna caratteristica nella versione iniziale del dataset.

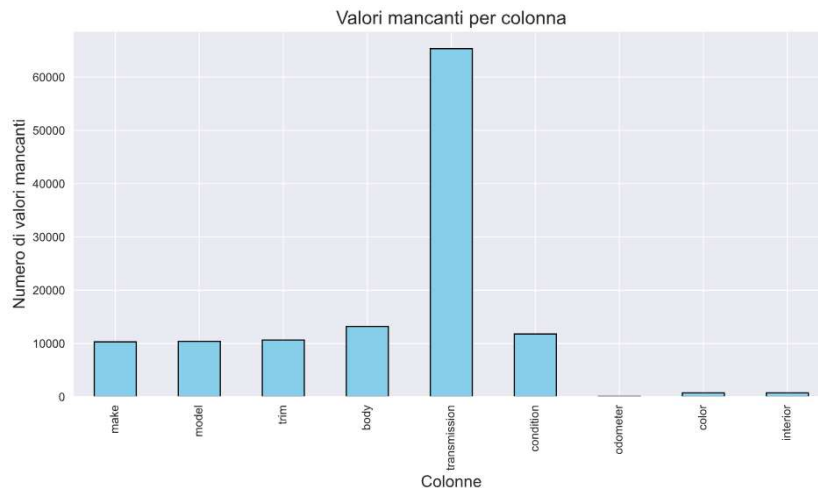


Figura 4: valori nulli per caratteristica.

La sfida, qui, non è tanto tecnica quanto di buon senso: riempire i vuoti senza introdurre bias. Per questo si è evitato l'uso indiscriminato di medie o mode globali, preferendo un'imputazione contestuale: per le variabili categoriche (es. carrozzeria, trasmissione) i valori mancanti sono sostituiti con la moda calcolata all'interno dello stesso modello di veicolo; per colorazione e colore interni la moda è calcolata per marca; per le variabili numeriche (condizione, chilometraggio) si utilizza la media calcolata per anno di produzione, assumendo che veicoli di età simile abbiano caratteristiche simili. Non sono stati riscontrati duplicati residui dopo le fasi di pulizia.

3.4 Numeri che il modello possa capire: encoding e scaling

Le variabili categoriche non possono essere utilizzate direttamente dai modelli di ML: vanno prima tradotte in numeri, ma nel modo giusto. Per la variabile trasmissione, binaria (manuale/automatico), è stato sufficiente un label encoding. Per le variabili categoriche ad alta cardinalità (marca, modello) è stato invece adottato il CatBoost Encoding, che integra i vantaggi del target encoding introducendo un parametro di regolarizzazione α che bilancia la statistica locale della categoria con la media globale del target. Questo riduce il rischio di overfitting sulle categorie rare rispetto a un target encoding classico, ed evita la crescita di dimensionalità tipica del one-hot encoding.

Per portare le variabili numeriche sullo stesso dominio si è applicata la standardizzazione (z-score normalization), evitando che le feature con scale più ampie (es. chilometraggio) monopolizzino l'apprendimento a scapito di quelle con range più piccoli (es. condizione).

3.5 Outlier: quando un dato è troppo per essere vero

Gli outlier possono distorcere in modo significativo l'apprendimento di un modello di ML, spingendolo ad adattarsi a casi estremi invece che al comportamento tipico dei dati. Sono state considerate outlier, e quindi rimosse, le osservazioni con un valore assoluto di z-score superiore a 3 sulle variabili numeriche rilevanti.

3.6 Feature engineering e selezione delle feature

Anziché utilizzare direttamente l'anno di produzione, si è scelto di calcolare l'età del veicolo come differenza tra l'anno di riferimento (2015) e l'anno di produzione: una trasformazione minima, ma che

rende la feature più direttamente interpretabile rispetto al prezzo. È stata inoltre applicata una selezione delle feature basata sulla varianza: le caratteristiche con varianza inferiore a una soglia prestabilita sono state rimosse, perché poco informative per il modello.

3.7 Il problema dello squilibrio tra classi

Per il task di classificazione il prezzo è stato discretizzato in cinque fasce (0–5.000€, 5.000–10.000€, 10.000–14.000€, 14.000–20.000€, 20.000€+), scelte per riflettere fasce di mercato intuitive piuttosto che quantili puramente statistici. L'analisi della distribuzione delle classi (Figura 5) ha però mostrato una disparità non trascurabile tra la classe maggioritaria e le classi minoritarie.

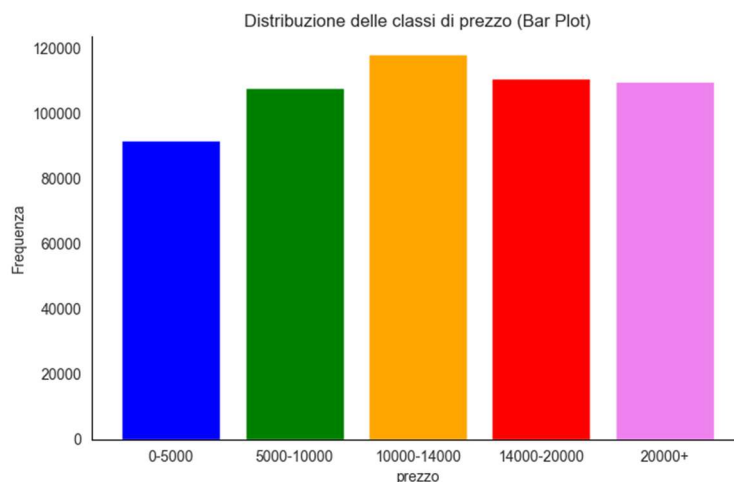


Figura 5a: distribuzione delle classi di prezzo prima del bilanciamento.

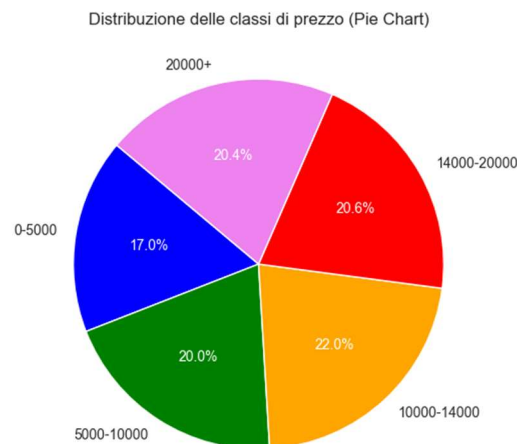


Figura 5b: incidenza percentuale delle classi di prezzo.

Uno squilibrio del genere, lasciato irrisolto, rischia di insegnare al modello la scorciatoia sbagliata: prevedere quasi sempre la classe più frequente e ignorare le altre. Per mitigarlo è stata adottata la tecnica di oversampling SMOTE (Synthetic Minority Over-sampling Technique), che genera campioni sintetici per le classi minoritarie interpolando tra osservazioni esistenti.

3.8 Perché il Random Forest?

La scelta del modello è una delle decisioni più importanti dell'intero progetto, ed è giusto spiegarne il perché e non limitarsi a darla per scontata. È stato scelto il RandomForest (Regressor per la regressione, Classifier per la classificazione) per la sua capacità di modellare relazioni non lineari tra le variabili senza dover assumere a priori una forma funzionale esplicita, come richiederebbe invece una regressione lineare. Il prezzo di un'auto è influenzato da molteplici fattori (anno, chilometraggio, marca, modello, condizione, ecc.) che interagiscono in modo tutt'altro che semplice da descrivere con una singola formula; un metodo di ensemble learning basato su alberi decisionali cattura queste interazioni tramite bootstrap sampling, addestramento di alberi su sottoinsiemi diversi con selezione casuale delle feature ad ogni split, e infine aggregazione dei risultati (media per la regressione, voto di maggioranza per la classificazione).

La scelta degli iperparametri è stata guidata dalla necessità di bilanciare accuratezza, costo computazionale e rischio di overfitting: `n_estimators=300`, `max_features=v_n_features`, `min_samples_split=10`, `min_samples_leaf=4`, `bootstrap=True`, `random_state=20`, `criterion=squared_error` per la regressione e `gini` per la classificazione. Il valore di `max_depth` è stato scelto analizzando l'andamento della validation loss/accuracy al crescere della profondità (Figura 6): `max_depth=18` per la regressione e `max_depth=20` per la classificazione, oltre i quali non si osservano miglioramenti significativi ma solo un maggior rischio di overfitting.

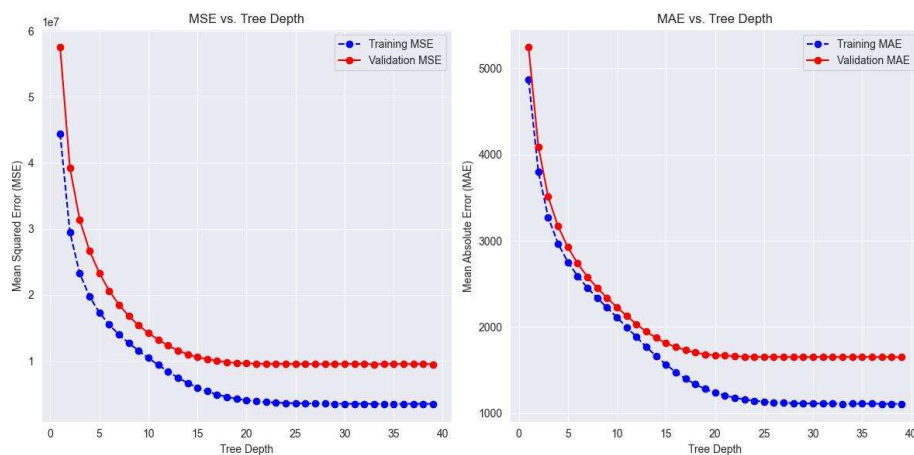


Figura 6a: andamento della validation loss (MAE/MSE) al variare di `max_depth` – regressione.



Figura 6b: andamento della validation accuracy al variare di max_depth – classificazione.

Non essendo obbligatorio l'uso dei soli modelli visti a corso, va segnalato che le tecniche di encoding (CatBoost) e di bilanciamento (SMOTE) qui adottate estendono il set di strumenti standard: sono state descritte nel dettaglio proprio per motivarne l'utilizzo e dimostrarne la corretta comprensione.

4. Analisi delle prestazioni

4.1 Come si valuta un modello, davvero

Valutare un modello su un singolo train/test split è comodo, ma rischioso: il risultato dipende troppo da quale particolare suddivisione dei dati è capitata. Per questo la valutazione è stata condotta con k-fold cross validation (k=10): ad ogni iterazione un fold diverso viene usato come test set, mentre le operazioni di preprocessing (imputazione, encoding, scaling, rimozione outlier, oversampling) vengono ricalcolate esclusivamente sul training set del fold corrente, per evitare che informazioni del test set “trapelino” nel training.

4.2 Risultati – task di classificazione

Le metriche standard derivate dalla matrice di confusione (precision, recall, f1-score, accuracy) sono riportate fold per fold in Tabella 2.

Fold	Accuracy	Precision	Recall	F1-score
1	0.7599	0.7594	0.7599	0.7589
2	0.7614	0.7605	0.7614	0.7600
3	0.7604	0.7595	0.7604	0.7592
4	0.7612	0.7605	0.7612	0.7601
5	0.7611	0.7606	0.7611	0.7602
6	0.7629	0.7623	0.7629	0.7618

Fold	Accuracy	Precision	Recall	F1-score
7	0.7609	0.7602	0.7609	0.7597
8	0.7647	0.7640	0.7647	0.7636
9	0.7613	0.7607	0.7613	0.7602
10	0.7626	0.7618	0.7626	0.7612

Tabella 2: metriche di validazione per il task di classificazione.

Le prestazioni sono stabili tra i fold (accuracy tra 0,7599 e 0,7647): un buon segnale di generalizzazione, non un caso fortunato su una singola suddivisione. È inoltre significativo che precision, recall e f1-score restino molto vicini all'accuracy in ogni fold: se una o più classi minoritarie fossero sistematicamente penalizzate, il recall medio si discosterebbe maggiormente. Questo è in buona parte un effetto atteso dell'oversampling SMOTE applicato in training.

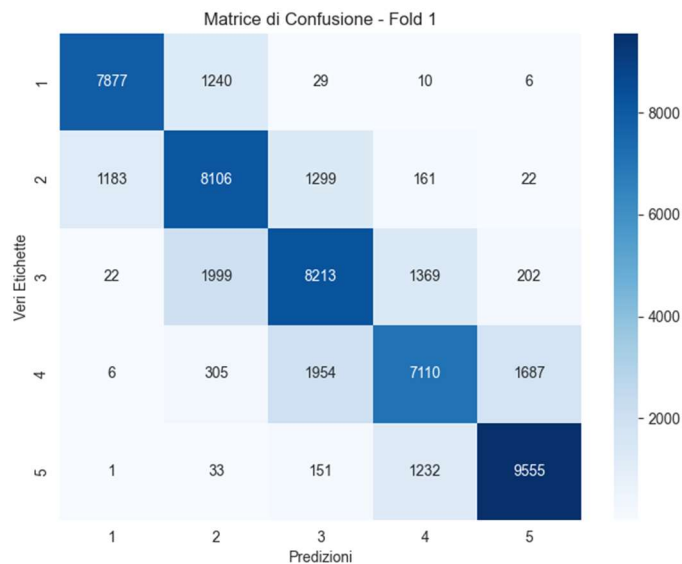


Figura 7a: matrice di confusione – primo fold.

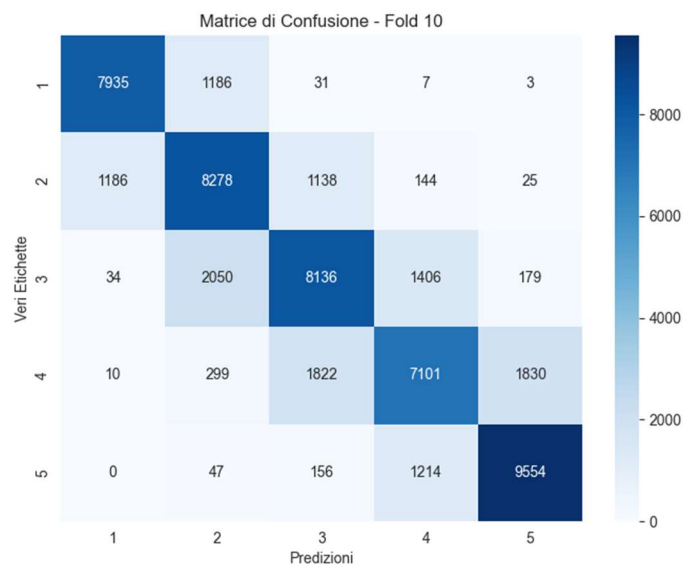


Figura 7b: matrice di confusione – decimo fold.

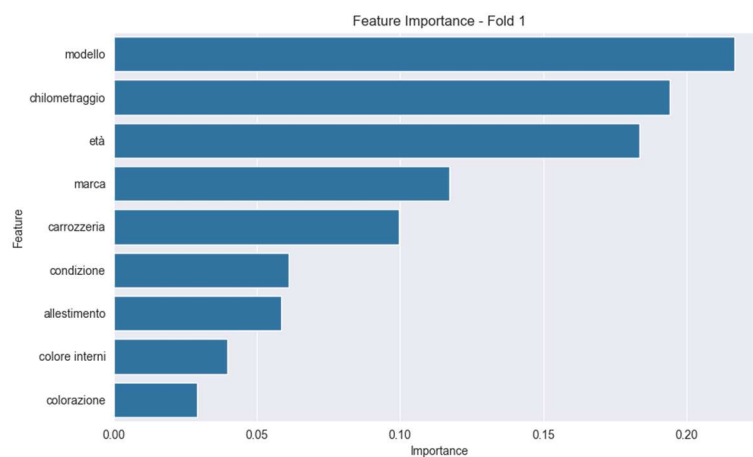


Figura 8: importanza delle caratteristiche – classificazione.

4.3 Risultati – task di regressione

Le metriche di errore per il task di regressione sono riportate fold per fold in Tabella 3.

Fold	MAE	MSE	RMSE	MAPE
1	1523.73	7924227.41	2815.00	16.7%
2	1550.91	8283304.95	2878.07	16.7%
3	1525.65	8973708.40	2995.61	16.8%
4	1530.21	7879336.22	2807.01	16.7%
5	1539.08	8816325.53	2969.23	16.5%
6	1549.62	8735598.15	2955.60	16.5%
7	1557.97	8581746.01	2929.46	16.8%

Fold	MAE	MSE	RMSE	MAPE
8	1544.62	8156220.90	2855.91	16.7%
9	1527.13	7820900.66	2796.59	17.0%
10	1531.88	8398450.02	2898.01	16.6%

Tabella 3: metriche di validazione per il task di regressione.

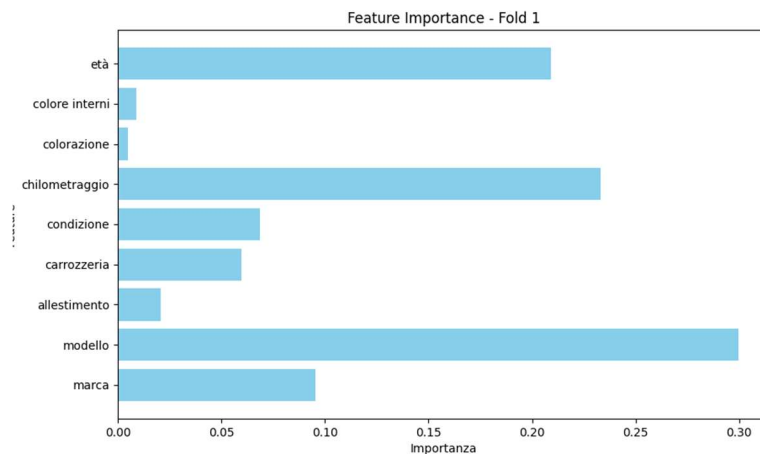


Figura 9: importanza delle caratteristiche – regressione.

4.4 Dove (e perché) il modello sbaglia

Le metriche aggregate raccontano solo metà della storia: per capire davvero un modello bisogna chiedersi anche dove e perché sbaglia, non solo quanto in media.

Classificazione

Le classi di prezzo utilizzate sono ordinali per costruzione, derivando dalla discretizzazione di una variabile continua. Ciò rende atteso che gli errori residui si concentrino soprattutto tra classi adiacenti (ad esempio tra la fascia 10.000–14.000€ e quella 14.000–20.000€): un'auto con prezzo vicino al confine tra due fasce può ragionevolmente finire scambiata con la fascia limitrofa. È invece molto meno probabile una confusione tra classi distanti (es. 0–5.000€ e 20.000€+), il cui profilo di feature (età, chilometraggio, condizione) è tipicamente molto diverso. Le matrici di confusione di Figura 7 permettono di verificare puntualmente questo pattern, osservando la concentrazione dei valori sulla diagonale principale e sulle sue immediate adiacenze.

Regressione

Confrontando MAE e RMSE in Tabella 3 emerge un rapporto RMSE/MAE prossimo a 1,85–1,95 in tutti i fold. Poiché l'RMSE penalizza quadraticamente gli errori più grandi mentre il MAE li pesa in modo lineare, un rapporto così superiore a 1 racconta qualcosa di preciso: la maggior parte delle predizioni ha un errore contenuto, ma esiste un sottoinsieme di veicoli (verosimilmente quelli con combinazioni di caratteristiche meno rappresentate nel dataset, come marche/modelli più rari o condizioni anomale) su cui il modello commette errori assoluti considerevolmente più grandi.

Il MAPE, attestato intorno al 16–17%, va letto insieme al MAE: a parità di errore assoluto in euro, l'errore percentuale è necessariamente più marcato sulle auto di fascia di prezzo più bassa, dove lo stesso scarto assoluto rappresenta una quota maggiore del valore del veicolo. In altre parole: il modello è relativamente più preciso in termini assoluti sui veicoli economici, ma relativamente meno preciso in termini percentuali proprio su quella stessa fascia.

5. Considerazioni finali e sviluppi futuri

Carval è partito da un problema molto concreto — quanto vale davvero un'auto usata — ed è arrivato a un sistema di Machine Learning che, sia in ottica di regressione sia di classificazione, produce stime stabili e generalizzabili. Le criticità tipiche di un dataset reale (rumore nelle categorie, valori mancanti, outlier, rischio di data leakage, sbilanciamento delle classi) sono state affrontate una per una con soluzioni motivate, e le scelte di modellazione (RandomForest, iperparametri, encoding, bilanciamento) sono giustificate dalle caratteristiche specifiche del problema, non scelte per default.

Detto questo, Carval non è un sistema perfetto, ed è giusto dirlo chiaramente. Sul fronte dei dati, l'integrazione di fonti più recenti e relative a mercati diversi da quello nord-americano lo renderebbe più generale, mentre informazioni più granulari (storico manutenzioni, incidenti, numero di proprietari precedenti) potrebbero affinare ulteriormente le stime. Sul fronte del modello, per vincoli di tempo non è stato possibile sperimentare in modo estensivo algoritmi alternativi (es. gradient boosting) né tecniche di ottimizzazione automatica degli iperparametri (es. algoritmi genetici o ricerca bayesiana): un naturale prossimo passo per spremere ulteriore accuratezza dal sistema.